

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	v
Índice de figuras	vi
Introducción	1
1 Marco teórico	4
1.1 ¿Que es un sistema operativo?	5
1.2 Tipos de sistemas operativos	7
1.2.1 Monousuario vs. Multiusuario	7
1.2.2 Monotarea vs. Multitarea	8
1.2.3 Tiempo real (RTOS)	9
1.2.4 Distribuidos	10
1.2.5 Móviles	10
1.2.6 Embebidos	11
1.2.7 Comparación entre tipos de Sistemas Operativos	12
1.3 Arquitecturas de Sistemas Operativos	14
1.3.1 Arquitectura Monolítica	14

1.3.2	Arquitectura en Microkernel	15
1.3.3	Arquitectura Hibrida	16
1.3.4	Arquitectura en Exokernel	17
1.3.5	Comparacion entre arquitecturas de Sistemas Operativos . . .	18
1.4	Componentes principales de un sistema operativo	20
1.4.1	Gestión de procesos	20
1.4.2	Gestión de memoria	24
1.4.3	Sistema de archivos	26
1.4.3.1	Estructura jerárquica	26
1.4.3.2	Gestión de datos persistentes	26
1.4.3.3	Operaciones de archivo	26
1.4.3.4	Asignación de espacio	27
1.4.3.5	Seguridad y atributos	27
1.4.3.6	Journaling	27
1.4.4	Gestión de dispositivos (E/S)	28
1.4.5	Interfaz de usuario	29
1.4.6	Seguridad y protección	30
1.4.6.1	Control de acceso	30
1.4.6.2	Aislamiento de procesos y protección de memoria . .	31
1.4.6.3	Prevención de intrusiones	31
1.4.6.4	Auditoría y registro	32
1.4.6.5	Modelos de seguridad clásicos	32
2	Revision de S.O. existentes	34

2.1	Windows	35
2.2	Mac OS	37
2.3	Linux	39
2.4	MINIX 3	41
2.5	RedoxOs	43
2.6	xv6	45
2.7	FreeRTOS	47
2.8	Fuchsia	49
2.9	Haiku	51
2.10	ToaruOS	52
3	Comparación técnica	54
	Referencias	56

Índice de tablas

1.1	Comparación de tipos de sistemas operativos basado mayormente en (Rawat, Vaibhav, 2025)	13
1.2	Comparación de arquitecturas de SO basado mayormente en (Harshvardhan and Irabatti, Shridhar, 2023)	19
1.3	Razones comunes para la creación de procesos en un sistema operativo (William Stallings, 2005)	22
1.4	Razones comunes para la terminación de procesos en un sistema operativo (William Stallings, 2005)	23
3.1	Comparación resumida de distintos sistemas operativos en orden de popularidad, elaborada en base a la investigación realizada en este proyecto.	55

Índice de figuras

1.1	Organización de nodos en un sistema operativo distribuido	10
1.2	Desarrollo cronológico de los sistemas operativos móviles y hitos tecnológicos asociados	11
1.3	Desarrollo cronológico de los sistemas operativos embebidos y hitos tecnológicos asociados	12
1.4	Sistema operativo basado en kernel monolítico	15
1.5	Sistema operativo basado en microkernel	16
1.6	Sistema operativo basado en kernel híbrido	17
1.7	Sistema operativo basado en exokernel	18
1.8	Diagrama de estados de un proceso	20
1.9	Traducción por paginación. Cada proceso (cajas turquesa) posee su propia tabla de páginas (columnas amarilla y rosada) que asigna páginas lógicas a marcos de la memoria física (bloques coloreados). Fuente: (Wikipedia, 2023d)	25
1.10	Evolución de las interfaces de usuario	29
2.1	Ícono de Windows	35
2.2	Ícono de macOS	37

2.3	Ícono de Linux	39
2.4	Ícono de MINIX 3	41
2.5	Ícono de RedoxOS	43
2.6	Ícono de xv6	45
2.7	Ícono de FreeRTOS	47
2.8	Arquitectura de FreeRTOS	48
2.9	Ícono de FuchsiaOS	49
2.10	Ícono de HaikuOS	51
2.11	Ícono de ToaruOS	52

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Marco teórico

1.1 ¿Que es un sistema operativo?

Según Stallings, el sistema operativo es el software encargado de controlar la ejecución de los programas y de administrar los recursos del procesador; además actúa como intermediario entre el usuario y el hardware, proporcionando los servicios necesarios para que las aplicaciones puedan ejecutarse correctamente (William Stallings, 2023)

Añadiendo esta idea, Tanenbaum y Bos explican que la finalidad del sistema operativo es convertir las interfaces de hardware, que suelen ser complejas e inconsistentes, en abstracciones practicas y sencillas de usar para los programadores y aplicaciones. Ademas de gestionar recursos como CPU, memoria y dispositivos de E/S, permitiendo así que las aplicaciones trabajen sin problemas con el hardware (Andrew S. Tanenbaum and Herbert Bos, 2023).

Según el artículo de la Revista Ogma, que es un científica y multidisciplinaria, un sistema operativo es el software principal que actúa como intermediario entre el usuario y el hardware, cuyo propósito es favorecer el uso eficiente de la computadora. Transforma las interfaces de hardware, complejas e inconsistentes, en abstracciones útiles y manejables para y aplicaciones y programadores, administra y mejora recursos (CPU, memoria, dispositivos de E/S y almacenamiento); organiza la ejecución concurrente de tareas, y ofrece espacios accesibles que permite el acceso de las computadoras, permitiendo que usuarios sin preparación instalen programas, administren archivos. (Cusme Vera, Ramona Johana and Fuentes Prado, Segundo Napoleón and Bravo Vega, Cinthya Dayana and Gualotuña Quinga, Guido Bolívar, 2022)

Un sistema operativo es como el sistema nervioso del computador, coordinando componentes hardware y software para que todo funcione de manera ordenada. Se

encarga de organizar y administrar recursos como procesador, memoria, dispositivos de E/S y almacenamiento.

1.2 Tipos de sistemas operativos

Los sistemas operativos pueden clasificarse de diversas maneras, por la administración de usuarios, administración de tareas, manejo de recursos, entorno de destino, entre otros criterios. En esta sección se abordarán específicamente las siguientes clasificaciones: sistemas operativos monousuario y multiusuario, monotarea y multitarea, de tiempo real (RTOS), distribuidos, móviles y embebidos. A continuación, el análisis de dichas clasificaciones.

1.2.1 Monousuario vs. Multiusuario

Estos tipos de sistemas operativos, son resultado de una clasificación por la cantidad de usuarios que los operan.

Monousuario: Como su nombre hace evidente, fue planificado para usar la computadora de forma que solo una persona a la vez lo haga correctamente, esto no significa necesariamente que solo pueda hacer una tarea a la vez (Kaur, Harpreet and Lata, Suman and Kaur, Harpreet, 2022). Sin embargo y completando el concepto, en la práctica más de una persona puede usarlo debido a que el sistema operativo no distingue entre personas (Olivares Cardenas, Paul Roberth, 2021). Según (Kaur, Harpreet and Lata, Suman and Kaur, Harpreet, 2022), algunos ejemplos resaltantes de sistemas operativos monousuario son Palm OS y Windows (versiones antiguas).

Multiusuario: Este tipo de sistema, hace posible que varios usuarios usen el computador de manera simultánea, para lo cual el sistema operativo debe administrar los recursos de manera adecuada; es necesario distinguir sistemas operativos multiusuario de sistemas monousuario compatibles con redes, en los cuales el único usuario “real” es

el administrador (Kaur, Harpreet and Lata, Suman and Kaur, Harpreet, 2022); otra característica del tipo multiusuario es distinguir entre las personas (usuarios) que acceden a la computadora (Olivares Cardenas, Paul Roberth, 2021). Según (Kaur, Harpreet and Lata, Suman and Kaur, Harpreet, 2022) complementado por (Olivares Cardenas, Paul Roberth, 2021) y (Kabiraj, Sumanta and Chandra, Subhashis Kumar and Gupta, Anjali, 2018), algunos ejemplos resaltantes de sistemas operativos multiusuario son Windows 7, Windows 10, UNIX, LINUX, VMS y el sistema operativo de servidor MVS.

1.2.2 Monotarea vs. Multitarea

Estos tipos de sistemas operativos, son resultado de una clasificación por la administración de la ejecución de tareas.

Monotarea: Como indica su raíz griega, solo hacen una tarea a la vez. En la actualidad están casi en completo desuso, debido a que no aprovecha de manera óptima los recursos. Un ejemplo resaltante de sistema operativo multitarea es MS-DOS, el cual actualmente se encuentra en desuso (Olivares Cardenas, Paul Roberth, 2021).

Multitarea: Estos sistemas operativos son capaces de ejecutar varias tareas a la vez, y por ello maximizan el uso de los recursos de la computadora. Además, también algunos sistemas operativos para móviles son multitarea. Los sistemas operativos multitarea, a su vez se pueden clasificar por su forma de administrar el tiempo de ejecución de sus aplicaciones en: Sistemas de tiempo compartido y sistemas de tiempo real (Olivares Cardenas, Paul Roberth, 2021). Según (Rawat, Vaibhav, 2025) algunos

ejemplos resaltantes de sistemas operativos multitarea son Windows 10/11, Linux, Unix, macOS.

1.2.3 Tiempo real (RTOS)

Los sistemas operativos de tiempo real son un tipo de sistema operativo multitarea. Mientras que su contraparte, los sistemas de tiempo compartido, se aprovechan del uso de pequeños intervalos de tiempo, para asignarlos a la ejecución de cada programa (time slice) y cambiar rápidamente entre las ejecuciones de estos programas (context switch), dando así la sensación de paralelismo; los sistemas operativos de tiempo real, no poseen intervalos de tiempo para definir la ejecución de un programa, sino que el programa se ejecuta hasta que termine (y dicho tiempo está definido también en el programa) o hasta que un programa de mayor prioridad lo interrumpa (Olivares Cardenas, Paul Roberth, 2021).

En la actualidad se usan mayormente los sistemas operativos de tiempo compartido, pero los sistemas operativos de tiempo real juegan un papel clave en campos que requieran precisión en cuanto al tiempo de procesamiento como medicina, tráfico aéreo, plantas de energía, entre otros (Olivares Cardenas, Paul Roberth, 2021).

En síntesis, estos sistemas buscan precisión en la sincronización, a partir de respuestas predecibles (determinismo) y se pueden clasificar de acuerdo a su tolerancia a errores en RTOS duros y RTOS suaves. Algunos ejemplos resaltantes son VxWorks, FreeRTOS y QNX (Rawat, Vaibhav, 2025).

1.2.4 Distribuidos

Los sistemas operativos distribuidos controlan múltiples nodos independientes como si fueran un solo sistema, varios computadores en red ejecutan conjuntamente un OS único, de modo que el usuario percibe una sola computadora con hardware extra. Cada nodo corre un núcleo mínimo y componentes de gestión que intercomunican y comparten recursos con el objetivo de lograr transparencia (single-system image) (Garg, Shweta and Vashist, Shrishti and Aggarwal, Shruti, 2013). Algunos ejemplos resaltantes son Google Fuchsia OS y Solaris OS (Rawat, Vaibhav, 2025).

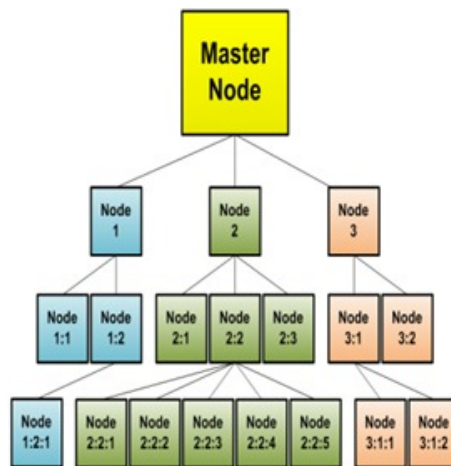


Figura 1.1: Organización de nodos en un sistema operativo distribuido (Garg, Shweta and Vashist, Shrishti and Aggarwal, Shruti, 2013)

1.2.5 Móviles

Los sistemas operativos móviles se diseñan para teléfonos inteligentes, tabletas y otros dispositivos portátiles táctiles, tiene como característica ser bastante intuitivos, ligeros y con un enfoque hacia el ahorro de la batería. Además, admiten funciones

avanzadas en tecnología como interacción táctil, comunicación inalámbrica y ejecución de diversas aplicaciones. Algunos de los ejemplos más resaltantes son Android OS e iOS (Rawat, Vaibhav, 2025).

A continuación, una línea de tiempo de la evolución de los sistemas operativos móviles:

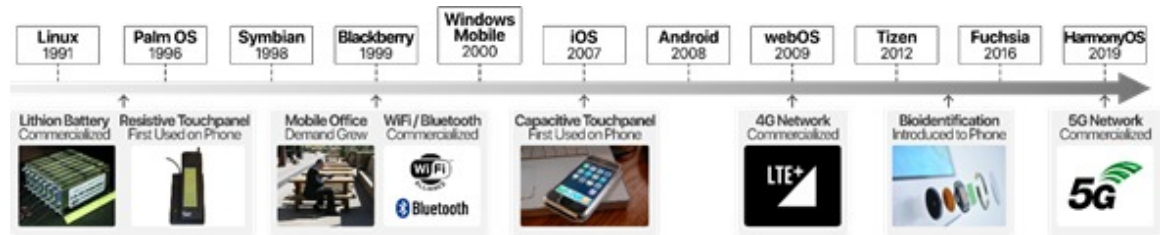


Figura 1.2: Desarrollo cronológico de los sistemas operativos móviles y hitos tecnológicos asociados (Jia, Shian and Wang, Xinbo and Song, Mingli and Chen, Gang, 2024)

1.2.6 Embebidos

Los sistemas operativos embebidos son diseñados para satisfacer los requerimientos específicos de los sistemas embebidos, son aplicados en diversos campos, tales como electrodomésticos, controles remotos e incluso instrumentos aeroespaciales. No requieren de demasiados recursos como energía, memoria ni espacio de almacenamiento; y aún así son capaces de brindar eficiencia y precisión, además de un nivel de personalización bastante alto para hacerlos funcionar en situaciones difíciles y cambiantes (Jia, Shian and Wang, Xinbo and Song, Mingli and Chen, Gang, 2024). Algunos de los ejemplos más resaltantes son Embedded Linux y Windows Embedded Compact (Rawat, Vaibhav, 2025).

A continuación, una línea de tiempo de la evolución de los sistemas operativos embebidos:

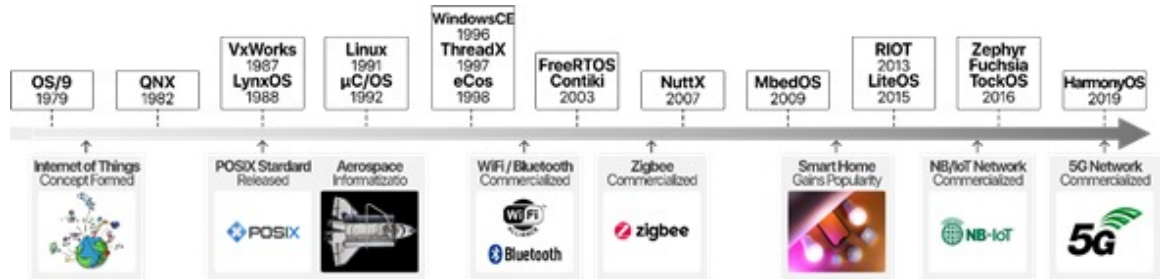


Figura 1.3: Desarrollo cronológico de los sistemas operativos embebidos y hitos tecnológicos asociados (Jia, Shian and Wang, Xinbo and Song, Mingli and Chen, Gang, 2024)

1.2.7 Comparación entre tipos de Sistemas Operativos

Tabla 1.1: Comparación de tipos de sistemas operativos basado mayormente en (Rawat, Vaibhav, 2025)

Tipo de SO	Ventajas	Desventajas
Monousuario	Simplicidad en diseño y administración, adecuado para uso personal	No diferencia usuarios, limitaciones en entornos colaborativos
Multiusuario	Permite que múltiples usuarios accedan simultáneamente, gestión de recursos más avanzada	Mayor complejidad en seguridad y administración de cuentas
Monotarea	Implementación sencilla, bajo consumo de recursos	No aprovecha el hardware moderno, ineficiente y obsoleto
Multitarea	Aprovecha mejor los recursos, permite ejecución paralela de procesos	Complejidad en la gestión de concurrencia y planificación
Tiempo Real (RTOS)	Respuestas deterministas, precisión crítica en aplicaciones sensibles (medicina, aeroespacial, etc.)	Difícil de implementar, requiere hardware confiable y costoso
Distribuidos	Transparencia de recursos, escalabilidad, tolerancia a fallos	Complejidad en diseño, comunicación entre nodos y sincronización
Móviles	Interfaz intuitiva, eficiencia energética, portabilidad	Limitados en recursos frente a sistemas de escritorio, dependencia de ecosistema cerrado
Embebidos	Alta eficiencia y personalización, bajos requerimientos de hardware	Poco flexibles, difíciles de actualizar y mantener

1.3 Arquitecturas de Sistemas Operativos

1.3.1 Arquitectura Monolítica

El sistema operativo monolítico es un sistema operativo muy simple donde el núcleo controla directamente la gestión de dispositivos, memoria, archivos y procesos. Todos los recursos del sistema son accesibles al núcleo. En los sistemas monolíticos, cada componente del sistema operativo está contenido dentro del núcleo.

En una arquitectura monolítica, el núcleo del sistema operativo está diseñado para proporcionar todos los servicios del sistema operativo, incluyendo la gestión de memoria, la programación de procesos, los controladores de dispositivos y los sistemas de archivos, en un único y gran binario. Esto significa que todo el código se ejecuta en el espacio del núcleo, sin separación entre los procesos del núcleo y los del usuario (GeeksforGeeks, 2024b).

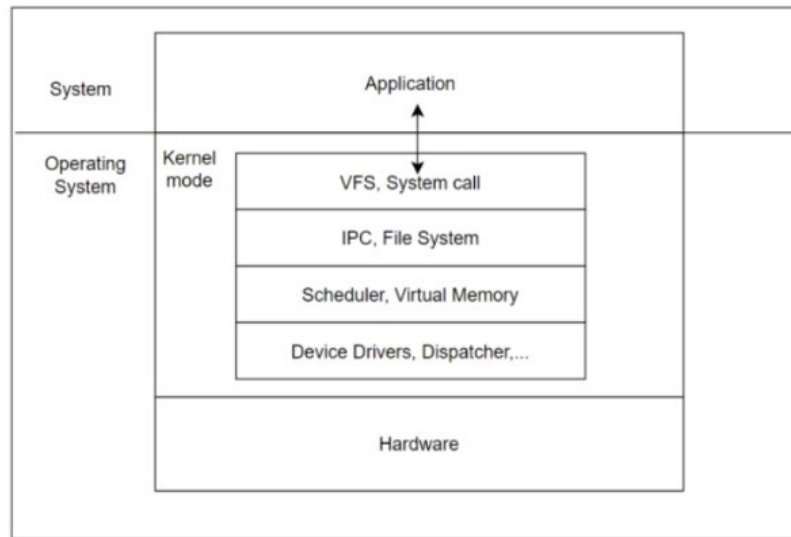


Figura 1.4: Sistema operativo basado en kernel monolítico (Harshvardhan and Irabatti, Shridhar, 2023)

1.3.2 Arquitectura en Microkernel

Un microkernel es un enfoque para diseñar un sistema operativo (SO). El microkernel proporciona servicios fundamentales para su funcionamiento, como la gestión básica de memoria, la programación de tareas. El microkernel es un tipo de sistema operativo que proporciona servicios básicos.

como los controladores de dispositivos y los sistemas de archivos, son gestionados por procesos a nivel de usuario. El proceso a nivel de usuario se comunica con el microkernel mediante el paso de mensajes. Esta forma de gestionar el proceso hace que los microkernels sean más modulares y flexibles que los kernels monolíticos tradicionales (GeeksforGeeks, 2024a).

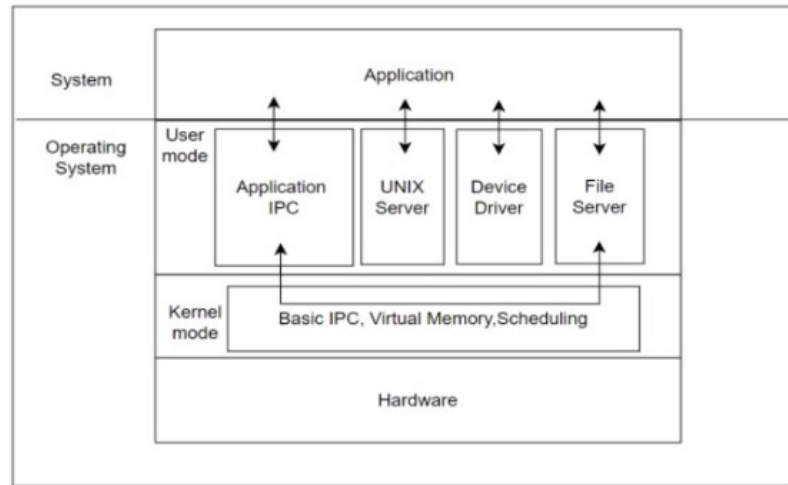


Figura 1.5: Sistema operativo basado en microkernel (Harshvardhan and Irabatti, Shridhar, 2023)

1.3.3 Arquitectura Híbrida

Un núcleo híbrido es una arquitectura de núcleo basada en la combinación de aspectos de las arquitecturas de micronúcleo y núcleo monolítico utilizadas en sistemas operativos .

La idea de esta categoría es tener una estructura de kernel similar a la de un microkernel, pero implementada en términos de un kernel monolítico. A diferencia de un microkernel, todos (o casi todos) los servicios del sistema operativo se encuentran en el espacio del kernel . Si bien no hay sobrecarga de rendimiento para el paso de mensajes ni el cambio de contexto entre el kernel y el modo de usuario, como en los kernels monolíticos , no hay ventajas de rendimiento al tener servicios en el espacio de usuario , como en los microkernels (Microsoft Wiki / Fandom, 2024).

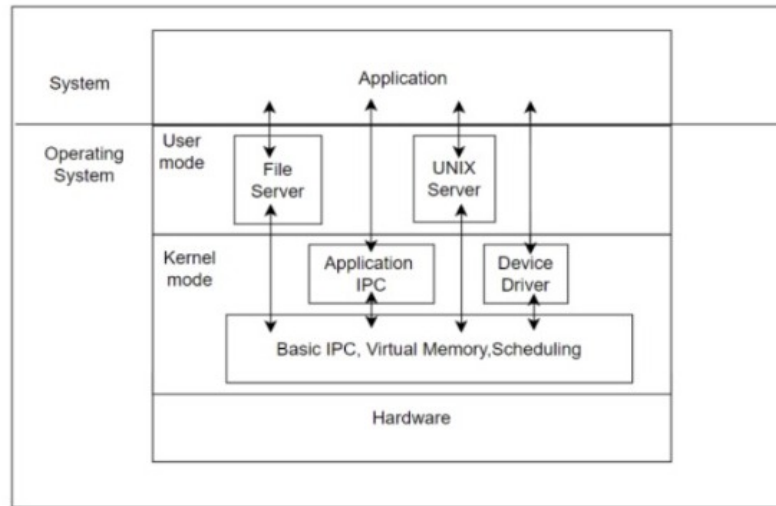


Figura 1.6: Sistema operativo basado en kernel híbrido (Harshvardhan and Irabatti, Shridhar, 2023)

1.3.4 Arquitectura en Exokernel

Exokernel es un sistema operativo desarrollado por el Instituto Tecnológico de Massachusetts (MIT) con el concepto de poner la aplicación bajo control. Los sistemas operativos Exokernel buscan proporcionar gestión de recursos de hardware a nivel de aplicación. La arquitectura de este sistema operativo está diseñada para separar la protección de recursos de la gestión, facilitando así la personalización específica de cada aplicación (Keetmalin, 2017).

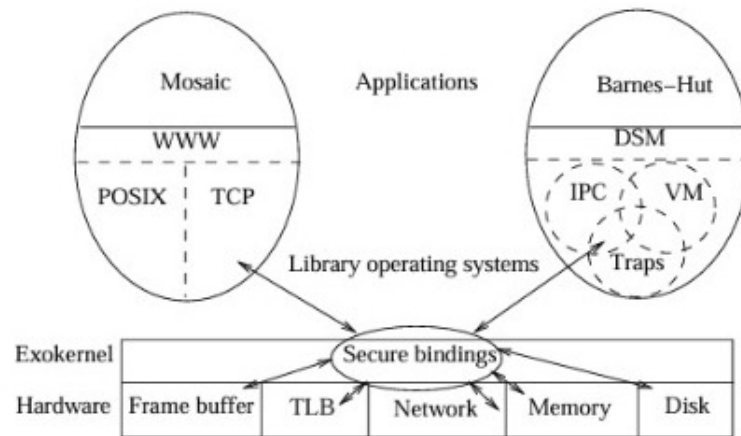


Figura 1.7: Sistema operativo basado en exokernel (Engler, Dawson R. and Kaashoek, M. Frans and O'Toole, James, Jr., 1995)

1.3.5 Comparacion entre arquitecturas de Sistemas Operativos

Tabla 1.2: Comparación de arquitecturas de SO basado mayormente en (Harshvardhan and Irabatti, Shridhar, 2023)

Arquitectura	Ventajas	Desventajas
Monolítica	Alto rendimiento, simple en diseño inicial	Difícil de mantener, poco modular
Microkernel	Modularidad, mayor seguridad	Mayor sobrecarga de comunicación
Híbrida	Combina rendimiento y modularidad	Complejidad de implementación
Exokernel	Máxima flexibilidad y control	Muy complejo, poco usado en producción

1.4 Componentes principales de un sistema operativo

Un sistema operativo integra diversos componentes que gestionan los recursos de hardware y ofrecen servicios a los procesos que se ejecutan en el sistema.

1.4.1 Gestión de procesos

La gestión de procesos es uno de los pilares fundamentales de cualquier sistema operativo, ya que se encarga de controlar la ejecución de programas y de garantizar un uso eficiente de la CPU. Un proceso, desde su creación hasta su finalización, pasa por diferentes estados a lo largo de su ciclo de vida. Al crearse, entra en el estado nuevo; luego pasa al estado preparado mientras espera acceso a la CPU. Cuando el planificador lo selecciona, pasa a ejecutando. Durante la ejecución, un proceso puede quedar en espera si requiere operaciones de entrada/salida, y finalmente alcanza el estado terminado al completar su tarea, como se muestra en la Figura 1.8.

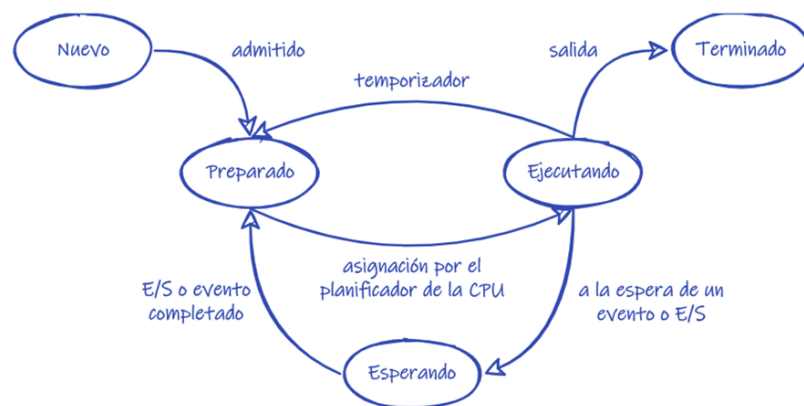


Figura 1.8: Diagrama de estados de un proceso. Fuente: (Jesus Torres, 2024)

Entre las funciones principales de la gestión de procesos se encuentra la creación y finalización de procesos. El sistema operativo se encarga de iniciar nuevos procesos mediante llamadas al sistema (por ejemplo, `fork` en Unix) y de eliminar aquellos que ya han concluido su ejecución (Juan Jose Costa Prats, 2015).

Otra función esencial es la planificación o *scheduling*, que consiste en decidir el orden en el que los procesos acceden a la CPU. Para ello se emplean algoritmos como FIFO, round robin o planificación basada en prioridades, según los objetivos de rendimiento y equidad del sistema (Juan Jose Costa Prats, 2015).

Asimismo, la gestión de procesos incluye mecanismos de sincronización y comunicación, conocidos como IPC (Inter-Process Communication). Estos permiten que los procesos se coordinen y compartan información de manera segura, evitando condiciones de carrera y conflictos al acceder a recursos comunes (Wikipedia, 2023b).

Tabla 1.3: Razones comunes para la creación de procesos en un sistema operativo
(William Stallings, 2005)

Tipo de proceso	Descripción
Proceso por lotes	El SO procesa trabajos en lote desde cinta/disco. Lee secuencias de control de trabajos automáticamente.
Sesión interactiva	Usuario inicia sesión desde un terminal.
Servicio del sistema	El SO crea procesos para servicios en segundo plano (ej: impresión) sin espera del usuario.
Proceso hijo	Proceso existente crea nuevos procesos para modularidad o paralelismo.

Tabla 1.4: Razones comunes para la terminación de procesos en un sistema operativo
(William Stallings, 2005)

Causa de terminación	Descripción
Finalización normal	Completación voluntaria mediante llamada al sistema.
Límite de tiempo	Exceso de tiempo de ejecución o espera.
Memoria no disponible	Memoria insuficiente o acceso no permitido.
Error de protección	Uso no autorizado de recursos.
Error aritmético	Operación inválida (ej: división por cero).
Fallo de E/S	Error en operación de entrada/salida.
Instrucción inválida	Ejecución de instrucción inexistente.
Instrucción privilegiada	Uso de instrucción reservada al SO.
Datos inapropiados	Datos erróneos o no inicializados.
Intervención externa	Terminación por operador o SO.
Terminación por padre	Finalización por proceso padre (automática o solicitada).

1.4.2 Gestión de memoria

La gestión de memoria es esencial para que el sistema operativo asigne y libere espacio en la memoria principal (RAM) a los procesos en ejecución. Su función principal consiste en aislar los espacios de direcciones de cada programa, de modo que puedan ejecutarse de manera concurrente sin interferir entre sí. Gracias a este control, la CPU puede cargar en memoria las instrucciones y datos necesarios de cada proceso en el momento oportuno. En términos generales, el administrador de memoria organiza los procesos de tal manera que se obtenga la máxima utilidad del espacio disponible, trasladando a la memoria principal la información que debe ejecutarse en cada instante (Wikipedia, 2023a).

En los sistemas modernos esta gestión se fundamenta en el concepto de memoria virtual, que proporciona a cada proceso una memoria lógica mayor que la memoria física disponible. Para ello se emplean técnicas como la paginación, en las que cada proceso dispone de su propia tabla de páginas que mapea direcciones lógicas hacia marcos de la memoria física. En la Figura 1.9 se ilustra este esquema, donde las páginas virtuales de los procesos se corresponden con bloques de memoria física compartida de manera ordenada.

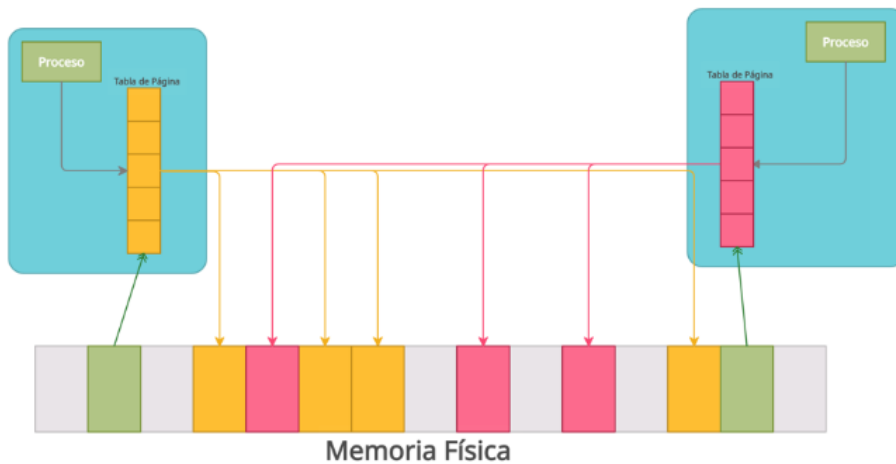


Figura 1.9: Traducción por paginación. Cada proceso (cajas turquesas) posee su propia tabla de páginas (columnas amarilla y rosada) que asigna páginas lógicas a marcos de la memoria física (bloques coloreados). Fuente: (Wikipedia, 2023d)

Entre las funciones principales del gestor de memoria se encuentran el control del uso de la memoria, la asignación y liberación de espacio para los procesos, la reubicación de datos en caso necesario, la gestión de la fragmentación y la compactación, la implementación de la memoria virtual y la protección de los espacios de direcciones para evitar accesos indebidos entre procesos. Estas tareas en conjunto garantizan un uso eficiente y seguro del recurso más limitado y compartido del sistema.

La traducción mediante paginación, como se aprecia en la figura, permite que cada proceso perciba su espacio de direcciones como contiguo, aunque en la práctica las páginas estén distribuidas en diferentes marcos físicos de la RAM. El sistema operativo se encarga de mantener actualizadas las tablas de páginas y, cuando es necesario, puede intercambiar páginas entre la memoria principal y el disco, asegurando así la abstracción de una memoria virtual continua (J. Cruz, 2020).

1.4.3 Sistema de archivos

1.4.3.1 Estructura jerárquica

El sistema de archivos organiza los archivos y directorios en forma de árbol invertido. A nivel lógico existe un único directorio raíz (`/`), nodo principal que contiene todos los demás. Cada nodo del árbol corresponde a un directorio que puede contener subdirectorios, archivos normales o especiales. Este diseño jerárquico con nombres únicos por directorio facilita la navegación y la gestión, evitando colisiones de nombres.

1.4.3.2 Gestión de datos persistentes

El sistema de archivos provee los medios para almacenar y recuperar datos de forma ordenada en la memoria secundaria (discos, SSD, etc.). La memoria se divide en bloques de tamaño fijo, y el sistema de archivos asigna a cada archivo los bloques necesarios. Además, controla la consistencia de los datos mediante técnicas como journaling, asegurando que la información persista aunque finalicen los procesos o falle el sistema. También gestiona la integridad ante fallos, permitiendo recuperar estructuras de disco sin pérdida de datos (R. Guirado, 2019).

1.4.3.3 Operaciones de archivo

El sistema de archivos ofrece llamadas básicas para crear, abrir, leer, escribir, renombrar y eliminar archivos y directorios. Por ejemplo, al invocar `open`, el kernel devuelve un descriptor de archivo que apunta a una entrada en la tabla de archivos abiertos y al inodo correspondiente. Según Bach (1986), esta tabla relaciona descriptores, entradas de acceso y estructuras de inodos internos. En otras palabras, cada proceso

maneja un descriptor representado como un entero, mientras que el sistema mantiene internamente una tabla que referencia los inodos de los archivos abiertos (R. Guirado, 2019).

1.4.3.4 Asignación de espacio

Los datos de cada archivo se almacenan en bloques de disco. El sistema de archivos controla qué bloques están libres y asigna los necesarios a cada archivo. En el esquema FAT (File Allocation Table), por ejemplo, cada disco dispone de una tabla con una entrada por bloque; cada entrada indica el siguiente bloque del archivo o un marcador especial que puede señalar bloque libre, bloque defectuoso o fin de archivo.

1.4.3.5 Seguridad y atributos

El sistema de archivos gestiona los atributos (metadatos) de cada archivo, así como la aplicación de permisos. En sistemas Unix, cada archivo se describe mediante un inodo que almacena permisos de acceso, propietario (UID/GID), fechas de modificación y tipo de archivo (Departamento de lenguajes y computacion, 2005).

1.4.3.6 Journaling

El journaling es un mecanismo que permite implementar transacciones en los sistemas de archivos. También se conoce como registro por diario, ya que mantiene un journal donde se almacena la información necesaria para restablecer los datos afectados en caso de fallo. Por ejemplo, ante un corte de energía, al reiniciar se reproduce el journal para completar o deshacer operaciones, restaurando así la consistencia. Gracias a este enfoque, los sistemas de archivos pueden volver rápidamente a producción con menor

riesgo de corrupción (Wikipedia, 2023c).

1.4.4 Gestión de dispositivos (E/S)

El sistema de entrada y salida del sistema operativo funciona como interfaz entre el hardware de los dispositivos (discos, teclados, impresoras, etc.) y el software, ocultando las particularidades de cada dispositivo al resto del sistema (Jesus Torres, 2024).

Sus componentes principales son:

- Controladores de dispositivo (drivers). Son programas específicos, normalmente provistos por el fabricante, que conocen los detalles del hardware y traducen las peticiones genéricas del sistema operativo en operaciones concretas sobre el dispositivo (Jesus Torres, 2024).
- Interfaz genérica de E/S. El sistema operativo ofrece llamadas estándar como `open`, `read`, `write`, `close` o `ioctl`, de modo que los programas interactúan con los dispositivos sin necesidad de conocer sus características físicas. En sistemas UNIX, todos los dispositivos de E/S se representan como archivos dentro del directorio `/dev`, lo que permite operaciones uniformes (Jesus Torres, 2024).
- Buffering y caching. El buffering compensa diferencias de velocidad durante transferencias de datos en curso, mientras que el caching se centra en la reutilización de datos accedidos recientemente, anticipando futuras solicitudes para acelerar accesos posteriores.
- Spooling. En dispositivos secuenciales no compartibles, como impresoras, el

sistema operativo encola los trabajos de varios procesos en un almacenamiento intermedio, gestionándolos en orden sin bloquear a los procesos que los enviaron (Jesus Torres, 2024).

En conjunto, la gestión de dispositivos permite a los procesos leer y escribir en hardware diverso mediante una interfaz uniforme, mientras el sistema operativo optimiza el flujo de datos y protege el acceso concurrente.

1.4.5 Interfaz de usuario

La interfaz de usuario es el medio a través del cual las personas interactúan con el sistema operativo.



Figura 1.10: Evolución de las interfaces de usuario (Wikipedia, 2016)

Las interfaces de usuario han evolucionado en diferentes formas de interacción:

- CLI (Command Line Interface). El usuario escribe comandos en una consola o terminal.
- GUI (Graphical User Interface). Interfaz gráfica con ventanas, iconos y menús, más intuitiva y exploratoria.
- NUI (Natural User Interface). Interacción más directa e intuitiva, basada en

mecanismos como el tacto, la voz o los gestos.

Un buen diseño de interfaz busca usabilidad e intuición, permitiendo al usuario dar órdenes al sistema operativo y visualizar información de estado. Esto incluye terminales, escritorios, menús de configuración o iconos de carpeta. El estilo de interfaz puede variar mucho entre sistemas, desde texto puro en algunos Unix básicos hasta entornos gráficos completos en sistemas modernos.

1.4.6 Seguridad y protección

La seguridad en un sistema operativo busca garantizar que solo los usuarios autorizados utilicen los recursos y lo hagan de la manera adecuada. Uno de los principios fundamentales es el de menor privilegio: cada proceso recibe únicamente los permisos estrictamente necesarios. La autenticación permite identificar al usuario, generalmente mediante contraseña, y asignar permisos correspondientes. En sistemas como Multics, un servicio de autenticación asocia cada proceso a un usuario; si este servicio falla, un proceso podría recibir permisos indebidos al quedar vinculado con un usuario equivocado (Trent Jaeger, 2008).

1.4.6.1 Control de acceso

El sistema operativo aplica un modelo de control de acceso que asigna permisos a los distintos recursos (archivos, dispositivos, regiones de memoria, etc.). Cada solicitud de un proceso se comprueba mediante un monitor de referencia, que valida las operaciones de acuerdo con los permisos asignados. De manera conceptual, esta verificación puede representarse como una matriz de acceso, donde las filas corresponden

a procesos o dominios, las columnas a objetos del sistema y las celdas a los permisos de lectura, escritura o ejecución.

En Unix, por ejemplo, cada archivo tiene bits de permiso y un propietario, siendo este último quien puede modificar los bits de acceso del archivo (Trent Jaeger, 2008).

1.4.6.2 Aislamiento de procesos y protección de memoria

El sistema operativo protege la memoria y el contexto de cada proceso para impedir interferencias entre ellos. Cada proceso se ejecuta en modo usuario (no privilegiado) dentro de su propio espacio virtual de direcciones. De esta manera, ningún proceso puede leer o escribir directamente en la memoria de otro, ni en la memoria del núcleo. La protección de memoria asegura que los datos de un usuario no puedan ser accedidos por otro y que el código crítico del kernel permanezca intacto.

Un ejemplo de este mecanismo es la arquitectura de privilegios o **rings**, donde las instrucciones sensibles solo se permiten en modo kernel (nivel 0). En este nivel privilegiado se ejecutan las operaciones críticas sobre memoria y dispositivos (Computer Science Department, 2018).

1.4.6.3 Prevención de intrusiones

Además de la protección interna, el sistema operativo incorpora mecanismos activos frente a amenazas externas o internas. Un sistema de prevención de intrusiones (IPS) supervisa el tráfico y la actividad en busca de comportamientos anómalos, ya sea mediante firmas conocidas o por detección de desviaciones. Si se identifica una amenaza, el IPS puede bloquearla antes de que cause daño, por ejemplo cerrando conexiones peligrosas o eliminando contenido malicioso.

Estos mecanismos suelen incluir filtros y cortafuegos integrados en el sistema operativo, que rechazan accesos no autorizados —ya sea por red o de manera local— según las políticas definidas. En conjunto, el sistema previene ataques como virus, intrusiones de red o escaladas de privilegios, al analizar cada petición de acceso y denegarla cuando coincide con patrones maliciosos (IBM, 2019).

1.4.6.4 Auditoría y registro

Con el fin de detectar incidentes y comportamientos sospechosos, el sistema operativo mantiene registros de auditoría de eventos relevantes: inicios y cierres de sesión, errores de autenticación, intentos fallidos de acceso, cambios en permisos, entre otros. Cada evento se almacena con la fecha, el usuario y la acción realizada, lo que permite revisar los logs para identificar accesos indebidos o intrusiones después de que ocurran.

Los sistemas de auditoría más avanzados incluso generan alertas automáticas al equipo de seguridad cuando detectan anomalías en el comportamiento (IBM, 2019).

1.4.6.5 Modelos de seguridad clásicos

En sistemas operativos con seguridad reforzada también se emplean modelos formales que definen con rigor las reglas de acceso. El modelo Bell–LaPadula se centra en la confidencialidad, aplicando el principio de “no leer hacia arriba, no escribir hacia abajo” (*read down, write up*). Con ello, un sujeto solo puede leer información de igual o menor nivel de clasificación y escribir en niveles iguales o superiores, evitando fugas de datos sensibles.

Por otro lado, el modelo Biba protege la integridad mediante las reglas opuestas:

“read up, write down”. Así se evita que un proceso de baja integridad contamine a uno de mayor integridad, o que datos críticos se vean corrompidos por información poco confiable.

En la práctica, estos modelos, junto con variantes como Clark–Wilson, proporcionan una base sólida para garantizar tanto la confidencialidad como la integridad en sistemas multigrado (Trent Jaeger, 2008).

Capítulo 2

Revision de S.O. existentes

2.1 Windows



Figura 2.1: Ícono de Windows (Wikimedia Commons contributors, 2021)

Windows es un sistema operativo desarrollado por Microsoft, cuyo origen se remonta a 1985 con la versión Windows 1.0, como una interfaz gráfica que extendía las funciones de MS-DOS. Desde entonces ha evolucionado hasta convertirse en una plataforma dominante en PCs, con múltiples versiones comerciales orientadas a usuarios domésticos, negocios y servidores (Alyssa Marie Rafael Natividad, 2023).

Windows posee una arquitectura monolítica, ya que gran parte del núcleo y los controladores E/S del dispositivo se procesan en modo kernel compartiendo la memoria. Windows es similar al sistema de UNIX, Windows tiene incluido una abstracción de hardware (HAL) y subsistema en modo usuario con el Win 32, POSIX, OS/2. (Mark E. Russinovich and David A. Solomon, 2005)

Está desarrollado en el lenguaje de programación C con sus componentes de bajo nivel como gestor de memoria, procesos, administración de E/S y partes de la interfaz del usuario y del sistema se implementaron en C++ y en ensamblador para las rutinas más sensibles, como arranque, gestión de interrupciones y entre otros (Mark E. Russinovich and David A. Solomon, 2005).

Entre sus componentes clave se encuentran el planificador de procesos y subprocesos que maneja procesos e hilos en modo kernel, el gestor de memoria virtual con paginación que protege cada proceso, subsistemas de archivos como NTFS, soporte de

red integral, sistema de drivers para dispositivos hardware, interfaz gráfica de usuario (GUI), componentes de seguridad como control de acceso, y el HAL para manejar diferencias entre arquitecturas de hardware (Justin E. Forrester and Barton P. Miller, 2000).

Windows tiene una comunidad muy grande de usuarios, desarrolladores y empresas. La documentación se publica mediante Microsoft Docs, los libros “Windows Internals” son referencias reconocidas, y Microsoft suele liberar guías técnicas, SDKs y APIs para desarrolladores. Sin embargo, el código fuente no es completamente abierto, lo que limita análisis y ediciones o posibles cambios en el código.

2.2 Mac OS



Figura 2.2: Ícono de macOS (GQ Informática, 2023)

El proyecto Mach dio comienzo en 1985 en la universidad de Carnegie Mellon con la finalidad de crear un microkernel que reemplace al BSD. Su diseño buscaba más seguridad para los usuarios, nose genero lo resultados esperados y se cancelo en 1994. Entonces el proyecto fue adoptado pro OSF/1 Y NeXTSTEP por lo que completo al combinar los componente de BSD, dando origen al kernel XNU de tipo monolítico, pero presento limitaciones con el System 7 y fracaso el proyecto, por ultimo este último avance fue adquirida por Steve Jobs, lo que permitió integrar NeXTSTEP y su kernel XNU como nuevo sistema operativo a Mac OS X, lanzado en 24 marzo del 2001 con el kernel de XNU, que integraba Mach 3.0 y el marco I/O kit, además de ser integrado en Intel Y ARM (Daan Keuper, 2012). El kernel de XNU prosee una arquitectura hibrida que tiene 4 componentes clave **Mach** que se encarga de administrar las tares, que contine hilos. Su característica principal es la mensajería, mediante puntos de comunicación (IPC), usa el Mach Interface Generator (MIG) para acortar lo procesos, tenemos tambien **BDS** que implementa procesos y señales UNIX, adema de sistema de archivos y redes, **I/O Kit** es el marco de controladores orientado a objetos y por ultimo **KEXTs** que encargar de enlazar no solo con el I/O Kit, sino tambien con

otras partes del kernel, extensión de red, sistema de archivos (Jonathan Levin, 2007). Varios de los primeros trabajos de software abierto de Apple son Darwin y WebKit. Darwin es un sistema operativo público que Apple introdujo en el año 2000. Este sistema fue liberado bajo la Licencia Pública de Apple (APSL), que ha sido aceptada por la OSI y la FSF (Jonathan Levin, 2007)

2.3 Linux



Figura 2.3: Ícono de Linux (Wikipedia, 2022)

Linux es un sistema operativo de tipo Unix iniciado en 1991 por Linus Torvalds como un kernel monolítico, modular y multitarea. Originalmente escrito para PCs i386, ha evolucionado hasta ofrecer compatibilidad POSIX y soporte extenso de hardware, siendo desarrollado bajo el modelo de código abierto, donde las mejoras provienen de contribuyentes individuales y corporativos, mientras que la dirección general la marca la comunidad y no un único proveedor (Wikipedia, 2022; Oracle latam, 2024). Su arquitectura se basa en un kernel monolítico moderno que integra características modulares. Todo el núcleo, incluidos los controladores de dispositivos, se ejecuta en un único espacio de direcciones en modo kernel. Sin embargo, soporta la carga y descarga dinámica de módulos en tiempo de ejecución, lo que le otorga flexibilidad similar a la de los micronúcleos sin sacrificar el rendimiento propio de un diseño monolítico (Robert Love, 2010, pp. 7). El núcleo de Linux está escrito principalmente en C, con secciones críticas en ensamblador para optimizar la interacción directa con el hardware. Las aplicaciones en espacio de usuario, en cambio, pueden desarrollarse en diversos lenguajes como C, C++, Python o Rust.

Entre sus componentes principales se encuentran la gestión de memoria con asignación y protección, la planificación de procesos que regula el uso de la CPU, los controladores de dispositivos para el hardware, el sistema de llamadas al sistema con sus mecanismos de seguridad, y el sistema de archivos virtual (VFS), encargado de unificar múltiples formatos y módulos adicionales (Oracle latam, 2024).

Linux posee una de las comunidades de código abierto más amplias y activas del mundo, integrada por miles de desarrolladores pertenecientes a empresas como Red Hat, Google, Intel o IBM, así como por colaboradores independientes. La documentación es igualmente extensa e incluye las **man pages**, el proyecto de documentación oficial del kernel (<https://docs.kernel.org/>), wikis como la Arch Wiki, foros especializados y numerosos libros técnicos.

2.4 MINIX 3



Figura 2.4: Ícono de MINIX 3 (Usenix, 2010)

MINIX fue creado en 1987 por Andrew Tanenbaum como herramienta docente para la enseñanza de sistemas operativos. En 2005 se anunció MINIX 3, una reimplementación centrada en la fiabilidad y la modularidad, cuyo propósito principal es ofrecer un sistema operativo compatible con POSIX, recuperable ante fallos y adecuado tanto para aplicaciones críticas como para entornos educativos. A diferencia de otros sistemas, fue escrito completamente desde cero, sin incluir código de AT&T, manteniendo compatibilidad con la versión 7 de Unix y los estándares POSIX (Usenix, 2010, pp. 10).

Su arquitectura está basada en un microkernel muy reducido que se encarga únicamente de interrupciones, gestión básica de procesos y comunicación mediante IPC. Todo lo demás se ejecuta en el espacio de usuario como servidores independientes: controladores de dispositivos, gestor de archivos, gestor de procesos y gestor de memoria. Este diseño multiserver permite que cada componente esté aislado, de modo que un fallo en un servidor no comprometa al sistema completo. Incluso cuenta con un servidor de reencarnación que reinicia automáticamente los componentes de-

fectuosos para mejorar la tolerancia a fallos (Jorrit N. Herder and Herbert Bos and Jakob Lichtenberg and Peter M. Chen and Andrew S. Tanenbaum, 2007).

MINIX 3 está escrito principalmente en C, con secciones en ensamblador dedicadas a la inicialización del hardware y a rutinas críticas de bajo nivel.

Entre sus componentes principales se encuentran la gestión de procesos, a cargo de un proceso supervisor que controla creación, terminación y recolección de fallos; la gestión de memoria, donde el microkernel provee segmentación y paginación básica mientras un servidor en espacio de usuario administra la memoria virtual; el servidor de archivos (Mini-FS) con soporte POSIX; controladores de dispositivos que se ejecutan como procesos de usuario aislados; y soporte para interfaces gráficas como X11 o entornos ligeros tipo EDE, siguiendo la tradición Unix (Usenix, 2010, pp. 12).

La comunidad de MINIX 3, aunque más pequeña en comparación con otros proyectos de código abierto, ha tenido impacto en el ámbito académico. Hasta 2010 su web oficial registraba cerca de 1.7 millones de visitas y más de 300,000 descargas de CD¹. El proyecto participó en Google Summer of Code entre 2008 y 2010, y mantiene recursos como wiki, grupos de discusión y repositorios en GitHub. Su documentación se apoya en el libro *Operating Systems: Design and Implementation* de Tanenbaum (3ª edición), artículos académicos y materiales de congresos, lo que lo convierte en una referencia educativa sólida en el campo de los sistemas operativos.

¹usenix.org

2.5 RedoxOs



Figura 2.5: Ícono de RedoxOS (Wikipedia contributors, 2025)

Redox es un sistema operativo tipo Unix que está escrito en el lenguaje de programación Rust, con el objetivo de implementar un microkernel y un sistema de aplicaciones. Rust es un lenguaje enfocado en la seguridad, rendimiento, gratuito y fácil de navegar. Redox se enfocó en la mejora de varios sistemas anteriores que presentaban errores. Uno de sus proyectos es Redox OS. Es compatible con POSIX, y la comunidad está trabajando para escribir la biblioteca libc en Rust, llamada relibc (Rohit Saini, 2018).

Dado que para los sistemas operativos es muy importante la seguridad, porque los sistemas operativos tienen un alto nivel de abstracción sobre los recursos del sistema, más aún en el caso de que Linux tuviera muchos errores en diferentes bibliotecas ocasionados por la seguridad de memoria. Rust en cambio evita todos esos problemas al tener seguridad de memoria en tiempo de compilación. El kernel de Redox contiene más de 20 mil líneas de código, y su diseño es de alto nivel por eso aún puede tener problemas (Simon Ellmann and Paul Emmerich, 2019).

El sistema operativo Redox OS se apoya en un microkernel y está inspirado en el sistema operativo MINIX. Las funciones tradicionales se implementan en kernels monolíticos, como controladores de dispositivo, pilas y el sistema de archivos. Los

microkernels son más seguros y ofrecen mayor estabilidad, pero una de las principales desventajas es que el rendimiento que ofrecen no está optimizado para la mayoría del hardware actual (Tomáš Ritter, 2019).

En cuanto a sus componentes clave, Redox OS integra un microkernel responsable de los procesos del sistema, un **sistema de archivos** implementado en espacio de usuario, **drivers** que también se ejecutan en espacio de usuario para incrementar la estabilidad, y compatibilidad POSIX que permite correr aplicaciones de otros sistemas Unix. También, añade **la biblioteca relibc**, escrita en Rust para mejorar la seguridad y la compatibilidad, además de **gestión de memoria** segura que evita vulnerabilidades comunes como los desbordamientos (The Redox OS Community, 2025).

2.6 xv6



Figura 2.6: Ícono de xv6 (Frans Kaashoek and Robert Morris, 2016)

xv6 es un sistema operativo educativo inspirado en Unix V6, desarrollado en 2006 por el MIT para su curso de sistemas operativos. Su propósito fue proporcionar un kernel moderno y simple que sustituyera al Unix V6 original, cuya complejidad dificultaba la enseñanza. xv6 sigue de cerca la estructura de Unix V6, pero está escrito en C ANSI y se ejecuta en arquitecturas Intel multiprocesador, convirtiéndose en una herramienta práctica y comprensible para la docencia (Frans Kaashoek and Robert Morris, 2016).

Su arquitectura corresponde a un kernel monolítico minimalista, en el que todo el núcleo del sistema operativo se ejecuta con privilegios de kernel, sin separación en servidores independientes. xv6 incluye soporte para multiprocesadores mediante bloqueos y estructuras internas de hilos, aunque carece de controladores de red o capacidades gráficas avanzadas. Sus componentes básicos abarcan la planificación de procesos con un esquema round-robin, la gestión de memoria con paginación simple, un sistema de archivos inspirado en Unix V6 y un conjunto reducido de llamadas al

sistema (Russ Cox and Frans Kaashoek and Robert Morris, 2022, pp. 25).

La implementación de xv6 está escrita principalmente en C, mientras que el código de inicio y las rutinas de interrupción se desarrollan en ensamblador x86. En sus versiones más recientes, también se encuentra una adaptación para la arquitectura RISC-V, lo que amplía su aplicabilidad en contextos educativos.

Entre los componentes principales destacan la gestión de procesos, que otorga a cada uno su propio espacio de direcciones, con un planificador round-robin y un cambio de contexto sencillo; la gestión de memoria mediante paginación básica y un asignador simple para el kernel; un sistema de archivos con inodos y directorios inspirado en Unix V6; y mecanismos elementales de sincronización como `sleep` y `wakeup`, que permiten estudiar las condiciones de carrera. Además, xv6 expone llamadas al sistema tradicionales de UNIX como `fork`, `exit`, `wait`, `open`, `read`, `write` y `close`, lo que facilita comprender la implementación exacta de estas primitivas fundamentales (Russ Cox and Frans Kaashoek and Robert Morris, 2022, pp. 26–30).

xv6 se ha consolidado como una herramienta académica ampliamente utilizada en universidades para cursos de sistemas operativos. Su código fuente está disponible públicamente, y los propios autores distribuyen un libro titulado *xv6: a simple, Unix-like teaching operating system*, que comenta línea por línea el código y explica tanto el funcionamiento como el diseño de sus estructuras y funciones. Gracias a su claridad y abundante documentación, xv6 se ha convertido en uno de los sistemas operativos de referencia para la enseñanza de fundamentos prácticos de diseño de kernels (Russ Cox and Frans Kaashoek and Robert Morris, 2022).

2.7 FreeRTOS



Figura 2.7: Ícono de FreeRTOS (Luis Llamas, 2025)

FreeRTOS es un sistema operativo en tiempo real con un kernel planificador, desarrollado para ejecutarse con microcontroladores. Está compuesto funcionalidades de planificación en tiempo real, comunicación entre tareas, análisis temporal y primitivas de sincronización con el propósito de cumplir con las tareas con el plazos estrictos de ejecución (Nannan He and Han-Way Huang, 2020). En cuanto a su arquitectura hace uso de un microkernel para administrar tareas en tiempo real, casi similar al RTLinux en que admiten núcleo dual, lo que permita separar tareas critica y no criticas (Anthony Serino and Liang Cheng, 2019) Esta escrito en lenguaje C estándar y con algunas lineas de código en ensamblador para que se acople a diferentes arquitecturas. Dentro de su componentes claves tenemos el **planificador**, **mecanismos de comunicación**, **sincronización** gracias al uso de colas y semáforos. Incluyendo la estructura de **Task Control Block (TCB)** para la administración de procesos y tareas (Richard Barry, 2018).

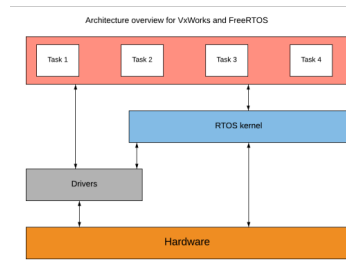


Figura 2.8: Diseño de la arquitectura de FreeRTOS (Luis Llamas, 2025)

FreeRTOS es código abierto con una licencia GPL, lo que permite su uso en aplicaciones comerciales. El código está disponible públicamente y cuenta con documentación extensa en el código fuente en su sitio oficial (The FreeRTOS Project, 2025).

2.8 Fuchsia



Figura 2.9: Ícono de FuchsiaOS (Fuchsia Project, 2025)

Fuchsia OS es un sistema operativo desarrollado por Google desde 2016, diseñado como una plataforma de propósito general para dispositivos embebidos, móviles, IoT y potencialmente de escritorio. A diferencia de Android, no se basa en Linux, sino en un núcleo propio denominado Zircon (Google Fuchsia Team, 2025b).

Su arquitectura está basada en un microkernel, en el que Zircon gestiona procesos, hilos, memoria y comunicación mediante objetos e IPC, mientras que servicios como sistemas de archivos y controladores se ejecutan en espacio de usuario, favoreciendo la seguridad y modularidad (Google Fuchsia Team, 2025a).

Fuchsia está escrito principalmente en C++, Rust y Dart. Zircon se implementa en C/C++, mientras que la capa de interfaz gráfica se desarrolla con Flutter (Dart), lo que facilita la portabilidad a múltiples plataformas (Google Inc., 2024).

Entre sus componentes clave se encuentran la gestión de procesos basada en objetos, memoria virtual con aislamiento estricto, soporte para múltiples sistemas de archivos (como MemFS, BlobFS y FAT) y un entorno gráfico con Flutter para experiencias multiplataforma (Google Fuchsia Team, 2025b).

El proyecto es mantenido principalmente por Google, con participación de la comunidad de código abierto. Su documentación oficial está disponible en Fuchsia.dev, que incluye guías técnicas, API y manuales de contribución, aunque parte del desarrollo se mantiene cerrado (Fuchsia Open Source Community, 2025).

2.9 Haiku



Figura 2.10: Ícono de HaikuOS(Haiku Project, 2025b)

Haiku es un sistema operativo de código abierto inspirado en BeOS, iniciado en 2001 con el objetivo de proporcionar un entorno moderno, rápido y sencillo de usar, orientado principalmente a equipos de escritorio (Haiku Project, 2025a).

Su arquitectura es híbrida, ya que utiliza un núcleo monolítico modular, pero con elementos que recuerdan a un microkernel, lo que equilibra rendimiento y flexibilidad (Haiku Project Developers, 2024). El sistema está desarrollado principalmente en C++, con partes en C y ensamblador para las interacciones de bajo nivel (Haiku Project Developers, 2024).

Entre sus componentes clave se incluyen un modelo de multitarea preventiva con planificación por prioridades, gestión de memoria virtual con paginación y aislamiento de procesos, y un sistema de archivos propio llamado OpenBFS, optimizado para indexación rápida (Haiku Project, 2023). Además, cuenta con una interfaz gráfica uniforme gestionada por el `app_server`.

Haiku es mantenido por la Haiku Project Foundation y su comunidad de desarrolladores voluntarios. El proyecto dispone de documentación oficial, foros y repositorios activos en GitHub, con lanzamientos beta periódicos que permiten probar y evaluar su madurez (Haiku Project Community, 2025).

2.10 ToaruOS

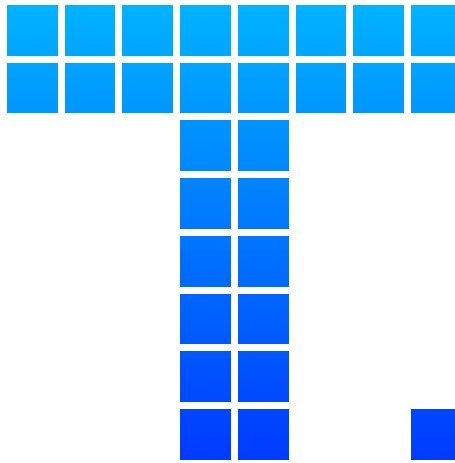


Figura 2.11: Ícono de ToaruOS (ToaruOS Project, 2025)

ToaruOS creado en 2010 por Kevin Lange, es un sistema operativo experimental con fines educativos, que busca evidenciar cómo se puede desarrollar un entorno completo desde la nada, independiente de kernels o librerías existentes. Su intención primigenia fue servir como soporte de investigación y aprendizaje para la comprensión del formato y funcionamiento interno de un sistema operativo moderno, incorporando subsistemas gráficos, de red y de usuario. A diferencia de proyectos como Linux o BSD, ToaruOS busca ser un entorno accesible para la enseñanza y la experimentación, y no un sistema de producción (K. Lange, 2025).

La arquitectura de ToaruOS es híbrida. Esto significa que el kernel es en gran medida monolítico por lo que, incluye directamente los módulos significativos del sistema, como la administración de memoria, procesos, controladores de dispositivos y el sistema de archivos, pero incluye características de microkernel como la posibilidad

de ejecución modular. Este diseño posibilita la integración y el rendimiento en un entorno práctico. El lenguaje de implementación es C, con cantidades inferiores en ensamblador para interrelaciones de bajo nivel con el hardware y el arranque (K. Lange, 2025).

Los elementos nucleares del sistema comprenden los elementos básicos de cualquier sistema operativo contemporáneo. Para la gestión de procesos pone en práctica tareas múltiples planificadas basada en prioridades, de igual manera de resistir hilos ligeros. Para la gestión de memoria abarca paginación, segmentación y asignación dinámica de memoria para los programas de usuario. Al respecto del sistema de archivos, dispone de su peculiar implementación denominada “tmpfs” para sistemas en memoria, asimismo la compatibilidad con EXT2 en versiones más actuales. Por otra parte, cuenta con un sistema gráfico completo, compuesto por un servidor de ventanas (Toaru Window Server), un gestor de ventanas y librerías gráficas que permiten ejecutar aplicaciones con interfaces gráficas, como un editor de texto o un navegador web básico (K. Lange, 2025).

La comunidad de ToaruOS es reducida en comparación con otros proyectos de código abierto, pero se mantiene activa en torno a su repositorio en GitHub, donde se documentan los cambios y se comparten implementaciones experimentales. La documentación disponible proviene principalmente de su creador, quien mantiene guías, el repositorio en Github y un blog explicativo sobre su evolución. Se ha mencionado grosso modo en limitadas ocasiones en artículos científicos de especialidad.

Capítulo 3

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Tabla 3.1: Comparación resumida de distintos sistemas operativos en orden de popularidad, elaborada en base a la investigación realizada en este proyecto.

Sistema	Arquitectura	Lenguaje	Kernel	HW	Documentación	Comunidad
Windows	Monolítica	C++	Grande	Amplio	Excelente	Masiva
XNU	Híbrida	C++	Grande	Intel	Buena	Apple
Linux	Monolítica	C	Grande	Amplio	Excelente	Activa
MINIX	Microkernel	C	Pequeño	Limitado	Buena	Académica
Redox	Microkernel	Rust	Mediano	Limitado	Moderada	Experimental
xv6	Monolítica	C	Pequeño	Limitado	Excelente	Académica
FreeRTOS	Microkernel	C	Minúsculo	Amplio	Extensa	Embebidos
Fuchsia	Microkernel	C++	Mediano	IoT	Buena	Google
Haiku	Híbrida	C++	Mediano	Limitado	Buena	Constante
ToaruOS	Híbrida	C	Pequeño	Limitado	Básica	Reducida

Referencias

Alyssa Marie Rafael Natividad (2023). Windows NT: A Comprehensive Analysis of its Role in Operating System Architecture. Preprint. Accedido el 21 de septiembre de 2025.

Andrew S. Tanenbaum and Herbert Bos (2023). *Modern Operating Systems*. Pearson, 5th edition.

Anthony Serino and Liang Cheng (2019). A Survey of Real-Time Operating Systems. Technical Report LU-CSE-19-003, Lehigh University, Department of Computer Science and Engineering, Bethlehem, PA. Accedido el 20 de septiembre de 2025.

Computer Science Department (2018). Protection and Virtual Memory. Accedido: 17 de septiembre de 2024.

Cusme Vera, Ramona Johana and Fuentes Prado, Segundo Napoleón and Bravo Vega, Cinthya Dayana and Gualotuña Quinga, Guido Bolívar (2022). Sistemas Operativos libres y Sistemas Operativos privativos: aplicaciones en el campo educativo. *Revista Científica Multidisciplinaria Ogma*, 1(3):85–97.

Daan Keuper (2012). XNU: a security evaluation. Master's Thesis, University of Twente. Accedido el 20 de septiembre de 2025.

Departamento de lenguajes y computacion (2005). Tema 4: sistema de archivos.

Accedido: 17 de septiembre de 2024.

Engler, Dawson R. and Kaashoek, M. Frans and O'Toole, James, Jr. (1995). Ex-
okernel: An Operating System Architecture for Application-Level Resource Man-
agement. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems
Principles (SOSP '95)*, pages 251–266, Cambridge, MA, USA. ACM.

Frans Kaashoek and Robert Morris (2016). Xv6, a simple Unix-like teaching operating
system. Accedido: 16 de septiembre de 2025.

Fuchsia Open Source Community (2025). Community Involvement in Fuchsia. Foros
y repositorios. Accedido: 20 septiembre 2025.

Fuchsia Project (2025). Fuchsia — Open Source Operating System. Sitio web oficial.
Accedido: 20 septiembre 2025.

Garg, Shweta and Vashist, Shrishti and Aggarwal, Shruti (2013). Distributed Operat-
ing System. *Journal of Harmonized Research in Engineering (JOHR)*, 1(2):95–96.
Received: October 2013; Accepted: November 2013.

GeeksforGeeks (2024a). Microkernel in Operating Systems. Accedido: 20 septiembre
2025.

GeeksforGeeks (2024b). Monolithic Architecture in Operating Systems. Accedido: 20
septiembre 2025.

Google Fuchsia Team (2025a). Fuchsia Concepts and APIs. Documentación técnica.
Accedido: 20 septiembre 2025.

Google Fuchsia Team (2025b). Zircon Kernel — Fuchsia Documentation. Documentación técnica. Accedido: 20 septiembre 2025.

Google Inc. (2024). Flutter and Dart in Fuchsia. Blog técnico de Google. Accedido: 20 septiembre 2025.

GQ Informática (2023). macOS, información completa. Accedido el 20 de septiembre de 2025.

Haiku Project (2023). OpenBFS — Be File System in Haiku. Documentación técnica. Accedido: 20 septiembre 2025.

Haiku Project (2025a). Haiku — Official Website. Sitio web oficial. Accedido: 20 septiembre 2025.

Haiku Project (2025b). Haiku — The Open Source Operating System. Sitio web oficial. Accedido: 20 septiembre 2025.

Haiku Project Community (2025). Haiku Community and Development. Foros y repositorios. Accedido: 20 septiembre 2025.

Haiku Project Developers (2024). Haiku Developer Documentation. Documentación técnica. Accedido: 20 septiembre 2025.

Harshvardhan and Irabatti, Shridhar (2023). Study of Kernels in Different Operating Systems in Mobile Devices. *Journal of Online Engineering Education*, 14(1s). Article received: 29 January 2023; Revised: 24 March 2023; Accepted: 20 April 2023.

- IBM (2019). ¿Qué es un sistema de prevención de intrusiones (IPS)? Accedido: 17 de septiembre de 2024.
- J. Cruz (2020). Tema 7: memoria virtual. Accedido: 17 de septiembre de 2024.
- Jesus Torres (2024). Sistemas Operativos. Online; accessed 20-June-2024.
- Jia, Shian and Wang, Xinbo and Song, Mingli and Chen, Gang (2024). Agent Centric Operating System – a Comprehensive Review and Outlook for Operating System. *arXiv preprint arXiv:2411.17710*.
- Jonathan Levin (2007). Inside the Mac OS X Kernel. In *Proceedings of the 24th Chaos Communication Congress (24C3)*. Accedido el 20 de septiembre de 2025.
- Jorrit N. Herder and Herbert Bos and Jakob Lichtenberg and Peter M. Chen and Andrew S. Tanenbaum (2007). The Architecture of a Reliable Operating System. Accedido: 16 de septiembre de 2024.
- Juan Jose Costa Prats (2015). Gestión de procesos. Accedido: 17 de septiembre de 2024.
- Justin E. Forrester and Barton P. Miller (2000). An Empirical Study of the Robustness of Windows NT Applications Using Random Testing. In *Proceedings of the 4th USENIX Windows Systems Symposium*, Seattle, WA, USA. Accedido el 21 de septiembre de 2025.
- K. Lange (2025). ToaruOS — Experimental Operating System. Repositorio oficial en GitHub. Accedido: 21 septiembre 2025.

- Kabiraj, Sumanta and Chandra, Subhashis Kumar and Gupta, Anjali (2018). Operating System: A Case Study. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, 2(3):167–168.
- Kaur, Harpreet and Lata, Suman and Kaur, Harpreet (2022). Operating System: A Review on Basic Concepts of Operating System. *International Journal of Scientific Research and Engineering Development*, 5(5):797–798.
- Keetmalin (2017). An Introduction on Exokernel Operating Systems. Blog post. Accedido: 20 septiembre 2025.
- Luis Llamas (2025). FreeRTOS Cheatsheet, la chuleta con lo imprescindible. Accedido el 20 de septiembre de 2025.
- Mark E. Russinovich and David A. Solomon (2005). *Microsoft Windows Internals, Fourth Edition*. Microsoft Press, 4th edition. Accedido el 21 de septiembre de 2025.
- Microsoft Wiki / Fandom (2024). Hybrid Kernel. Accedido: 20 septiembre 2025.
- Nannan He and Han-Way Huang (2020). USE OF FreeRTOS IN TEACHING A REAL-TIME EMBEDDED SYSTEMS DESIGN COURSE. *Computers in Education Journal*, XI(2). Accedido el 20 de septiembre de 2025.
- Olivares Cardenas, Paul Roberth (2021). Sistemas Operativos y Hardware: Clasificación, Características y Virtualización. Monografía, Universidad Nacional de Educación Enrique Guzmán y Valle, Facultad de Ciencias. Examen de Suficiencia Profesional, Resolución N. 1627-2021-D-FAC, para optar al título de Licenciado en Educación, Lima, Perú.

Oracle latam (2024). ¿Qué es Linux? Accedido: 16 de septiembre de 2024.

R. Guirado (2019). Tema 7: sistema de archivos. Accedido: 17 de septiembre de 2024.

Rawat, Vaibhav (2025). A brief study on Operating Systems along with their history, types, working, challenges and advancements. *Zenodo*, pages 12–15. Chandigarh Group of Colleges, Jhanjeri, Punjab, India. Department of Computer Science and Engineering.

Richard Barry (2018). *Mastering the FreeRTOS Real Time Kernel - A Hands-On Tutorial Guide*. Real Time Engineers Ltd. For FreeRTOS V10.0.0. Accedido el 20 de septiembre de 2025.

Robert Love (2010). *Linux Kernel Development*. Addison-Wesley Professional, Third edition.

Rohit Saini (2018). Priority Based Scheduler in Rust Based Redox Operating System. Master's thesis, Indian Institute of Technology Madras. Accedido el 20 de septiembre de 2025.

Russ Cox and Frans Kaashoek and Robert Morris (2022). xv6: a simple, Unix-like teaching operating system. Accedido: 17 de septiembre de 2025.

Simon Ellmann and Paul Emmerich (2019). Porting ixy.rs to Redox. Technical report, Technical University of Munich. Accedido el 20 de septiembre de 2025.

The FreeRTOS Project (2025). FreeRTOS - Market leading RTOS for embedded systems. Accedido el 20 de septiembre de 2025.

The Redox OS Community (2025). The Redox OS Book. Accedido el 20 de septiembre de 2025.

ToaruOS Project (2025). Logo de ToaruOS. Imagen PNG en sitio oficial. Accedido: 21 septiembre 2025. Licencia NCSA / University of Illinois.

Tomáš Ritter (2019). Disk Encryption in Redox OS. Master's thesis, Masaryk University, Faculty of Informatics. Accedido el 20 de septiembre de 2025.

Trent Jaeger (2008). Operating system security. Accedido: 17 de septiembre de 2024.

Usenix (2010). MINIX 3: status report and current research. Accedido: 16 de septiembre de 2024.

Wikimedia Commons contributors (2021). Logotipo y marca denominativa de Windows - 2021. Archivo SVG en Wikipedia. Subido el 17 de octubre de 2021. Basado en el logotipo oficial de Microsoft Windows 11.

Wikipedia (2016). Archivo:CLI-GUI-NUI, evolución de interfaces de usuario. Accedido: 17 de septiembre de 2024.

Wikipedia (2022). Nucleo linux. Accedido: 17 de septiembre de 2024.

Wikipedia (2023a). Gestion de memoria. Accedido: 17 de septiembre de 2024.

Wikipedia (2023b). Inter-process communication. Accedido: 17 de septiembre de 2024.

Wikipedia (2023c). Journaling. Accedido: 17 de septiembre de 2024.

Wikipedia (2023d). Tabla de paginacion. Accedido: 17 de septiembre de 2024.

Wikipedia contributors (2025). Redox (operating system) — Wikipedia, The Free Encyclopedia. [Online; accessed 20-September-2025].

William Stallings (2005). *Sistemas operativos, aspectos internos y pricipios de diseño*. pearson prentice hall, fifth edition.

William Stallings (2023). *Computer Organization and Architecture: Designing for Performance*. Pearson, 12th edition.